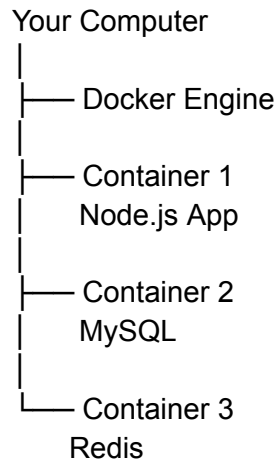


Part 1 – Docker Basics (Containers)

What is Docker?

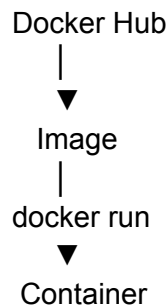
Docker is a platform that lets you package an application together with everything it needs (Node.js, Python, libraries, OS files, etc.) into a **container**.

Think of it like this:



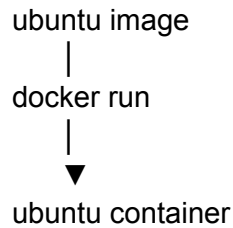
Each container is isolated from the others.

Docker Architecture



- **Docker Hub** → online repository of images
- **Image** → blueprint/template
- **Container** → running instance of an image

Example:



You can create many containers from the same image.

Check Docker Version

`docker --version`

Example

Docker version 28.2.2

See Docker Information

`docker info`

Shows:

- running containers
 - images
 - storage
 - CPUs
 - memory
-

Download an Image

`docker pull nginx`

Docker downloads the latest nginx image.

Specific version:

`docker pull nginx:1.27`

If no version is provided:

latest

is downloaded.

List Images

docker images

Example

REPOSITORY	TAG	IMAGE ID
nginx	latest	a12345
node	22	b56789

Run a Container

docker run nginx

Docker will

1. Search locally
 2. Download if missing
 3. Create container
 4. Start container
-

Interactive Mode

Run Ubuntu

docker **run** -it ubuntu

Inside the container:

ls

pwd

apt update

Exit:

exit

Detached Mode (-d)

Normally:

docker run nginx

The terminal becomes attached to nginx.

Instead:

docker run -d nginx

Now nginx runs in the background.

List Running Containers

docker ps

Example

CONTAINER ID

IMAGE

STATUS

PORTS

NAMES

Only running containers.

List All Containers

`docker ps -a`

Includes

- Running
 - Stopped
 - Exited
-

Stop Container

`docker stop container_name`

Example

`docker stop happy_cat`

or

`docker stop 95df34`

Start Existing Container

`docker start container_name`

Restart

`docker restart container_name`

Remove Container

`docker rm container_name`

Container must be stopped.

Force remove:

`docker rm -f container_name`

Port Mapping (-p)

Without mapping:

Container Port 80

Host cannot access it.

With mapping

`docker run -p 8080:80 nginx`

Meaning

Host:8080



Container:80

Open

`http://localhost:8080`

Multiple Ports

`docker run -p 3000:3000 -p 9229:9229 image`

Environment Variables (-e)

Many images require configuration.

Example

`docker run -e MYSQL_ROOT_PASSWORD=123456 mysql`

Another example

`docker run -e PORT=5000 myapp`

Inside the container
PORT=5000

Container Name

Instead of random names

docker **run --name web** nginx

Now

docker **stop** web

instead of

happy_lion

Execute Commands Inside Running Container

docker **exec -it web** bash

Now you're inside the container.

View Logs

docker **logs** web

Follow logs

docker **logs -f** web

Remove Images

docker **rmi** nginx

Common **docker run** Options

Option	Description
<code>-d</code>	Detached mode
<code>-it</code>	Interactive terminal
<code>-p host:container</code>	Port mapping
<code>-e KEY=VALUE</code>	Environment variable
<code>--name</code>	Container name
<code>--rm</code>	Delete container after exit

Example

Run nginx:

```
docker run -d -p 8080:80 --name web nginx
```

Example – Running a MySQL Container

1. Pull the Image

```
docker pull mysql:8.4
```

or simply

```
docker pull mysql
```

2. Run the Container

```
docker run -d --name mysql-db -p 3306:3306 -e MYSQL_ROOT_PASSWORD=123456
mysql:8.4
```

Explanation

Option	Meaning
<code>-d</code>	Run in the background (Detached Mode)
<code>--name mysql-db</code>	Give the container a readable name
<code>-p 3306:3306</code>	Map host port 3306 to container port 3306
<code>-e MYSQL_ROOT_PASSWORD=123456</code>	Set the MySQL root password
<code>mysql:8.4</code>	Image and version to run

3. Verify the Container is Running

```
docker ps
```

Example output

CONTAINER ID	IMAGE	STATUS	PORTS	NAMES
9c8f0d8b5d1a	mysql:8.4	Up 20 seconds	0.0.0.0:3306->3306/tcp	mysql-db

4. View Container Logs

MySQL takes a few seconds to initialize.

```
docker logs mysql-db
```

Follow the logs continuously:

```
docker logs -f mysql-db
```

When you see something similar to:

MySQL init process done.

Ready for start up.

the database is ready.

Connect to the Container Shell

A container is like a tiny Linux machine.

To open a shell inside the running container:

```
docker exec -it mysql-db bash
```

Now your terminal changes to something like:

```
root@9c8f0d8b5d1a:/#
```

You are now inside the container.

Explore the Container

```
pwd  
ls  
cd /var/lib/mysql  
ls
```

These are the actual MySQL database files.

Exit the shell:

```
exit
```

Connect to the MySQL Client

Instead of opening a shell first, you can start the MySQL client directly:

```
docker exec -it mysql-db mysql -u root -p
```

Enter the password:

```
123456
```

Now you're inside MySQL:

```
mysql>
```

Basic SQL Commands

Show databases:

```
SHOW DATABASES;
```

Create a database:

```
CREATE DATABASE school;
```

Use it:

```
USE school;
```

Create a table:

```
CREATE TABLE students (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100)  
);
```

Insert data:

```
INSERT INTO students (name)  
VALUES ('Moshe');
```

Read data:

```
SELECT * FROM students;
```

Exit MySQL:

```
exit
```

Stop the Container

```
docker stop mysql-db
```

Start It Again

```
docker start mysql-db
```

Restart It

`docker restart mysql-db`

Remove the Container

First stop it:

`docker stop mysql-db`

Then remove it:

`docker rm mysql-db`

Complete Workflow

Pull Image



`docker pull mysql:8.4`



Run Container



`docker run -d --name mysql-db -p 3306:3306 -e MYSQL_ROOT_PASSWORD=123456
mysql:8.4`



List Running Containers



`docker ps`



View Logs



`docker logs -f mysql-db`

